Correctly rounded floating-point maths functions

Document number:
P3864R1

Date:
2026-02-22

Audience:
SG6

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Guy Davidson <gd@6it.dev>
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3864/github

ABSTRACT: This paper proposes adding five overload sets to the standard library for addition, subtraction, multiplication, division, and square root calculation, correctly rounded as specified in ISO/IEC 60559:2020.

Contents

1. Introduction

Floating-point maths functions are rounded after calculation. The rounding mode used is part of the floating-point state which is maintained per-thread. This introduces two problems:

1. Changing rounding modes, for example for calculations that require correct rounding in a set of optimized expression evaluations, is unergonomic.
2. There is a burden on library writers to restore the floating-point state to the condition it was in when a library function was called.

This is not a new problem, and the C standard in Annex F reserves the prefix `cr_` for functions fully matching the [60559:2020] mathematical operations; see 7.33.8 [Mathematics `<math.h>`]:

> Function names that begin with `cr_` are potentially reserved identifiers and may be added to the `<math.h>` header. The `cr_` prefix is intended to indicate a correctly rounded version of the function.

This paper proposes adding five overload sets to the standard library for addition, subtraction, multiplication, division and square root calculation.

These functions guarantee that the operation will be carried out as if with infinite precision, and rounded using the “roundTiesToEven” rounding mode, as specified in [60559:2020]. The parameter type must satisfy the type trait `std::numeric_limits<T>::is_iec559`.

This solves problem 1 directly by providing a more ergonomic way of expressing intention: the client can explicitly state that they require correctly rounded calculation without having to ensure that the floating-point state is set appropriately.

This does not solve problem 2 directly, since this does not necessarily affect the floating-point state at all. However, it reduces the sources of error when correct rounding is important.

1.1. Related papers

Proposal [P3375R3] seeks to introduce reliable reproducibility to floating-point operations regardless of platform. This proposal partially addresses this problem by reducing the places where implementations can diverge. However, it does not necessarily solve the divergence introduced by inlining or optimization choices, which are declared per translation unit.

This can be mitigated, at the cost of performance, by creating a `struct` which contains a value of the appropriate type as a single member, and implementing the arithmetic operators in a separate library, linked at runtime, in terms of the correctly rounded functions. In this way, a client can mix correctly rounded and optimized operations in a single translation unit.

This author currently believes that more performant reproducibility can only be achieved by introducing a new type which is defined to be immune to the floating-point optimizations specified in [60559:2020].

2. Design considerations

2.1. Error handling

The error handling for the proposed functions should match that of the existing mathematical functions. That is:

- NaN is propagated.
- A range errors occur on overflow.
- Infinity may be propagated without error.
- A pole error occurs when dividing by zero.
- Underflow or denormalization is ignored.
- A domain error occurs when the result is not mathematically defined.

2.2. `cr_sqrt` for -0 arguments

If the argument to `cr_sqrt` is negative zero, the result should also be negative zero. This matches the behavior of `sqrt` in typical implementations (although this behavior is not mandated by the C standard), and it matches the behavior specified for **squareRoot**(-0) in [60559:2020].

3. Wording

The proposed changes are based on [N5014].

[version.syn]

In [version.syn], insert a feature-test macro as follows:

```
#define __cpp_lib_math_cr_functions 20XXXXL // also in <cmath>
```

EDITOR'S NOTE: The name is based on `__cpp_lib_math_special_functions`.

[cmath.syn]

In [cmath.syn], prior to the declaration of the mathematical special functions, insert the following:

```
// [c.math.crfunc], correctly rounded functions
constexpr iec-559-type cr_add(iec-559-type x, iec-559-type y) noexcept;
constexpr iec-559-type cr_sub(iec-559-type x, iec-559-type y) noexcept;
constexpr iec-559-type cr_mul(iec-559-type x, iec-559-type y) noexcept;
```

```
constexpr iec-559-type cr_div(iec-559-type x, iec-559-type y) noexcept;
constexpr iec-559-type cr_sqrt(iec-559-type x) noexcept;
```

Amend [cmath.syn] paragraph 1 to read:

> The contents and meaning of the header `<cmath>` are a subset of the C standard library header `<math.h>` and only the declarations shown in the synopsis above are present, with the addition of
>
> - a three-dimensional hypotenuse function ([c.math.hypot3]),
> - a linear interpolation function ([c.math.lerp]), ~~and~~
> - **the correctly rounded functions ([c.math.crfunc]), and**
> - the mathematical special functions described in [sf.cmath].

EDITOR'S NOTE: The use of bullets is added here.

Amend [cmath.syn] paragraph 2 to read:

> For each function with at least one parameter of type *`floating-point-type`*, the implementation provides an overload for each cv-unqualified floating-point type ([basic.fundamental])**; for each function with at least one parameter of type *`iec-559-type`*, the implementation provides an overload for each cv-unqualified floating-point type `T` for which `numeric_limits<T>::is_iec559` is `true`.** ~~where all~~ **All** uses of *`floating-point-type`* **or *`iec-559-type`*** in the function signature are replaced with ~~that~~ **the provided** floating-point type.

§ [c.math.crfunc]

Immediately preceding [cf.cmath], introduce a new subclause:

Correctly rounded functions [c.math.crfunc]

¶ The correctly rounded functions *correctly round* the result of an operation, meaning that the result is calculated as if using infinite precision, then rounded using "roundTiesToEven", as specified in ISO/IEC 60559:2020.

¶ If any argument value to any of the functions below is a NaN (Not a Number), the function shall return a NaN but it shall not report a domain error. A range error occurs if all arguments are finite and the result is too large in magnitude to be represented in the destination type.

¶ Unless otherwise specified, each function is defined for all finite values, for negative infinity, and for positive infinity.

```
constexpr iec-559-type cr_add(iec-559-type x, iec-559-type y) noexcept;
```

¶ *Returns*: $x + y$ correctly rounded.

```
constexpr iec-559-type cr_sub(iec-559-type x, iec-559-type y) noexcept;
```

¶ *Returns*: $x - y$ correctly rounded.

```
constexpr iec-559-type cr_mul(iec-559-type x, iec-559-type y) noexcept;
```

¶ *Returns*: $x \times y$ correctly rounded.

```
constexpr iec-559-type cr_div(iec-559-type x, iec-559-type y) noexcept;
```

¶ *Returns*: $\frac{x}{y}$ correctly rounded.

¶ *Remarks*: A pole error occurs if `y` equals zero.

```
constexpr iec-559-type cr_sqrt(iec-559-type x) noexcept;
```

¶ *Returns*: $\sqrt{x}$ correctly rounded. If `x` is -0, the result is also -0.

¶ *Remarks*: A domain error occurs if `x` < 0.

EDITOR'S NOTE: The introductory wording is loosely inspired by [sf.cmath.general].

§ 4. References and bibliography

[N5014] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-08-05)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf

[60559:2020] *ISO.* ISO/IEC 60559:2020 — Information technology — Microprocessor Systems — Floating-Point arithmetic (2025-05)
https://www.iso.org/standard/80985.html

[P3375R3] *Guy Davidson.* Reproducible floating-point results (2025-05-12)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3375r3.html